

Java

WORA : write once, read anytime.

Author: James Gosling.

Initial name of Project: OAK lang.

1995: Sunmicro system now **Oracle** has its own java.

Java is a **platform independent** due to **JDK**.

JIT: just in time compiler.

Less time complexity due to **JIT**.

- Object oriented language.
- platform independent.
- portable and scalable.
- secured and robust.
- Supports mobile applications.
- Multithreading (multiple processes at a time).
- Memory management.

Class name and file name should be the same.

For compile: `javac class.java` (checks file name in local pc)

for running: `java class_name` (checks public class name in code)

`Public static void main (string args[]){     };`

Deep dive into public static void main(String args[]);

Changing Modifier Order:

- `static public void main(String args[])` is VALID. (You can swap `public` and `static`).

Array Bracket Placement:

- `public static void main(String[] args)` is VALID.
- `public static void main(String []args)` is VALID.
- `public static void main(String args[])` is VALID.

Varargs (Variable Arguments):

- `public static void main(String... args)` is VALID (Introduced in Java 5).

Argument Name:

- `public static void main(String[] xyz)` is VALID. The name `args` is standard, not mandatory.

`public static final void main(String... args)` is VALID.

`public static synchronized void main(String... args)` is VALID.

`public static strictfp void main(String... args)` is VALID.

Can the main method be abstract? A: No. Runtime Error.

Args[] array length:

- **Default Value:** Empty Array object (`{ }`)
- **Default Length:** 0
- **Is it Null?:** No

```
public static void main(String[] args) {
    System.out.println(args[0]);
}
```

-> `ArrayIndexOutOfBoundsException`

Java memory model.

Stack : method calling, reference of objects, local variables.

- Bottom to top.

Heap(Expandable memory): All the objects are stored in heap.

- Top to Bottom.

Data Type	Size in Bits	Size in Bytes	Important MCQ Note
byte	8 bits	1 byte	Smallest integer type.
short	16 bits	2 bytes	Rarely used, but good for saving memory in large arrays.
int	32 bits	4 bytes	The default type for integral numbers (e.g., 5 is an int).
long	64 bits	8 bytes	Requires 'L' suffix (e.g., 500L).
float	32 bits	4 bytes	Single precision. Requires 'f' suffix (e.g., 5.5f).
double	64 bits	8 bytes	The default type for decimals (e.g., 5.5 is a double).
char	16 bits	2 bytes	Unsigned (no negative values). Java uses Unicode, not ASCII.

boolean : changes behaviour.

java .lang: automatically gets called. It contains.....

1. **Object class**: Parent class of all classes.

2. **String, StringBuffer, StringBuilder**: Classes for manipulating text.
3. **Wrapper Classes: Integer, Double, Boolean** (Turn primitives to objects)
4. **Thread & Runnable**,
5. **System**: Contains standard input, output, and error streams (**System.out**, [System.in](#)).

[java.io](#): handles all input (reading data) and output (writing data)

- File inputs team(**Images, videos**), file output stream, file reader(**reading characters**), file writer file class.
- **Serialization**: The process of converting an object into a stream of bytes to save it to a file. (Interface: **Serializable**).

Java.util package: collections framework.

- **List (Interface)**: Ordered, allows duplicates (e.g., **ArrayList, LinkedList**).
- **Set (Interface)**: Unordered, **NO** duplicates (e.g., **HashSet**).
- **Map (Interface)**: Key-Value pairs (e.g., **HashMap**)
- Calendar, date_time, scanner, Random.

Summary:

Package	Full Name	Primary Purpose	Do you need to import it?	Key Classes (Memorize these)
java.lang	Java Language	Core logic, basic types, system access.	NO (Automatic)	String, System, Math, Object, Thread
java.io	Java Input/Output	Reading/Writing files and streams.	YES	File, InputStream, FileReader, Serializable
java.util	Java Utility	Data structures (Lists/Maps), Date, Input parsing.	YES	ArrayList, HashMap, Scanner, Date

HashCode in java: Default memory location but can't access it.

Feature	== (Operator)	.equals() (Method)
---------	---------------	--------------------

Type	It is a binary operator .	It is a method defined in the <code>Object</code> class.
Primitive s	Used to compare values (e.g., <code>5 == 5</code>).	Cannot be used on primitives (int, char, etc.).
Objects	Compares memory addresses (references). It checks if two variables point to the <i>exact same object</i> in memory.	Designed to compare content (logical equality).
Default Behavior	Always reference comparison.	In the <code>Object</code> class, it behaves exactly like <code>==</code> (reference check) unless the class overrides it.

In blockchain hashcodes can be given to other locations(hashing).

public String toString() method:

1. Returns a "string representation" of the object.
2. If you do not override then output -> (by default if does not print content) **ClassName@HexadecimalHashCode**.
 - To get proper output it must be overridden in class.
 - **Printing an Object:** When you write `System.out.println(s)`, Java implicitly changes it to `System.out.println(s.toString())`

- If the object is `null`, it prints the string "null" (it does **not** throw an error).

Default values for primitives in java: (only for instance and static vars)

- Local variables needs to be initialised during declaration.

Data Type	Default Value	MCQ Note
byte	0	
short	0	
int	0	
long	0L	Note the L suffix.
float	0.0f	Note the f suffix.
double	0.0d	Usually printed just as 0.0.
char	'\u0000'	This is the Unicode Null Character. It is distinct from 0 or null.
boolean	false	It is never true or null.

Imp : Array elements get default value to elements as 0 initially.

Array Declaration in Java:

```
int[] arr;  
int arr[];  
int []arr;  
int[] a, b; two 1d arrays  
int[5] arr; -> compiletime error.
```

Multidimensional array declaration:

```
int[][] arr;  
int arr[][];  
int[] arr[];  
int[][] arr[];  
int[] a, b[]; a->1d_array, b-> 2d_array.  
int[] arr = new int[5]; correct way.  
Trap: int[] arr = new int[]; $\\rightarrow$ Compile Error.
```

`new int[3]{10, 20, 30};` Compile Error.

Trap: You cannot specify the size inside the brackets if you provide the values.

`new int[-5];`:- NegativeArraySizeException.

Ascii values:

Character Type	Range of Characters	ASCII / Unicode Range	Formula to Remember
Digits	'0' to '9'	48 to 57	0 is 48

Uppercase	'A' to 'Z'	65 to 90	A is 65
Lowercase	'a' to 'z'	97 to 122	a is 97

'a' (97) - 'A' (65) = 32

MCQ Tip: If you have 'F' and want 'f', just add 32.

System.out.println('A' + 'B'); -> 65 ('A') + 66 ('B') = 131.

' '(space) = 32 always

Architecture of java:

To write the code:

JVM is the engine inside. (compilers and debuggers, javadoc)

Jvm is platform dependent and acts as a translator.

Jvm contains JIT(turbocharger).

To run the code:

JRE contains the JVM + Libraries.(minimum requirement)

JDK contains the JRE + Development Tools.

JIT compiler:

First the interpreter works line by line then JIT starts its work.

Thread intercommunication: wait(), notify(), notifyall().

clone():

Before use must implement the cloneable interface.

protected method.

finalize(): for garbage collection.

Final method: cannot be overridden.

Type casting in java:

1. **Implicit/widening casting:** small to large. Int to float.
2. **Explicit/narrowing:** Done forcefully (data loss may happen).
3. String to any other d_type conversion is easy.
 - **.parseInt():** to convert into int("12a"-> numberformat exception). Returns primitive.
 - **<wrapper class>.valueOf():** number to string. Returns object.
 - **Integer.toString(int i):** alternative to above.
int a = Integer.valueOf("123"); correct.
Wrapper classes can contain null but primitives cant.
They have diff methods to use through object class.

Feature	Autoboxing	Unboxing
Direction	Primitive $\rightarrow$ Wrapper Object	Wrapper Object $\rightarrow$ Primitive
Action	Java wraps a primitive value into an object automatically.	Java extracts the value from an object automatically.
Example	<code>Integer a = 10;</code>	<code>int b = a;</code>
Behind Scenes	Compiler calls <code>Integer.valueOf(10)</code>	Compiler calls <code>a.intValue()</code>

In unboxing NullPointerException happens.

Enum constructors are always private.

String: Use it when your data will **not change** (e.g., Error messages, Database URLs).

StringBuffer: Use it when you need to modify text in a **Multi-threaded** environment (Legacy/Safety).

StringBuilder: Use it when you need to modify text in a **Single-threaded** environment (Speed/Standard choice).

The Two Ways to Create a Thread (MCQ Favorite):

1. Extending the **Thread** Class
2. Implementing the **Runnable** Interface (Recommended)(can extend another class)

5 states of threads:

New: create new thread but haven't started it.

Runnable(): waiting for cpu to pick it up.(already started).

Running: the executing code.

blocked/ waiting: blocked.

Terminated: dead. Run method finished.

Trap: If you call **start()** twice on the same thread object, it throws

IllegalThreadStateException

start(): creates a new stack and calls the run method.

run(): It comes into the running state. Contains the code that needs to be executed.

sleep(): pause the current thread. Throws Interrupted exception.

join(): Waits for a thread to die(completes its work).

yield(): Hints thread scheduler that current thread is willing to pause. Current thread goes from running to tunable state.

Interthread communication: wait(), notify(), notifyall()-> better for deadlock not to happen.

Reflection in Java is a powerful API (Application Programming Interface) that allows a program to inspect, examine, and modify the behavior of classes, interfaces, fields,

1. Inspection
2. Manipulation
3. Bypassing Encapsulation.

Lambda function: no name, no return type declaration, and no modifiers.

`Runnable r2 = () -> System.out.println("Hello from Lambda");`

Multiple parameters:

`(a, b) -> System.out.println(a + b);`

The Trap: If you use the `return` keyword, you **must** use curly braces `{}` and a semicolon `;`

JDBC:Fact: In JDBC `ResultSet`, the column index starts at 1.

`rs.getString(0)` throws `SQLException`.

- `rs.next()` : call this at least once to go to the first row else get `SQLException`.
- The type-4 driver used most nowadays.
- Reverse order of closing the connection: `ResultSet` -> `Statement` -> `Connection`.